

Capítulo 23: Tooling de Ingeniería y Arquitectura (sbt y Modularidad)

1. Explicación Teórica: El Dominio del Build Tool y el Desacoplamiento

El ecosistema de ingeniería de Scala se define por el uso de herramientas de construcción (*build tools*) que gestionan dependencias, compilación y empaquetado, siendo **sbt (Scala Build Tool)** la elección histórica para proyectos de escala industrial. Recientemente, **Scala CLI** ha emergido como la herramienta moderna por defecto para *scripts* y prototipos.

Modularidad y sbt

Para proyectos grandes y complejos, la modularidad se implementa dividiendo el código en **múltiples submódulos o subproyectos**. sbt gestiona esta arquitectura, permitiendo que la compilación se ejecute en **paralelo** entre los diferentes módulos, lo cual es crucial para reducir los tiempos de compilación de grandes bases de código. Los proyectos de Scala suelen estructurarse en paquetes que corresponden a diferentes capítulos o áreas de la aplicación.

Tooling de Calidad y Estándares

Además de las herramientas de compilación, los ingenieros de Scala utilizan *tooling* avanzado para garantizar la calidad del código:

1. **sbt**: Gestiona la estructura de directorios, dependencias y tareas avanzadas.
2. **Scala CLI / Coursier**: Herramientas modernas de instalación (`cs`) y *scripting* ligero.
3. **Metals / IntelliJ IDEA**: Entornos de desarrollo, siendo **Metals** la opción ligera preferida para Scala 3 y **IntelliJ IDEA** la opción para grandes bases de código.
4. **Scalafix**: Una herramienta poderosa para *linting* y refactorización, utilizada para **imponer estándares de código** y automatizar refactorizaciones masivas.

Modularidad Arquitectónica y Desacoplamiento

La verdadera ingeniería de software implica **reducir el acoplamiento** entre módulos. El estilo de Scala, impulsado por la Programación Funcional Pura (PFP), busca escribir código débilmente acoplado. Un patrón clave para lograr esto es el **Principio del Mínimo Conocimiento (Law of Demeter)**:

- Un módulo debe depender solo de la **interfaz mínima** (un *trait* o *boundary*) que necesita de otro módulo, en lugar de depender de la implementación completa de ese módulo.

- Esto previene que un cambio en una implementación subyacente de un módulo A fuerce una **recompilación en cascada** innecesaria de un módulo B, si B solo dependía de la interfaz abstracta.

2. Sintaxis: sbt y Declaración de Dependencias

A. Configuración Básica de sbt (build.sbt)

El archivo `build.sbt` define los ajustes y dependencias del proyecto. Para incluir dependencias y configurar la versión de Scala, se utiliza la sintaxis específica del DSL (Domain Specific Language) de sbt.

Sintaxis (build.sbt):

```
1  lazy val root = project
2    .in(file(".")) // Proyecto raíz en el directorio actual
3    .settings(
4      // Versión de Scala (ej. para Scala 3)
5      scalaVersion := "3.3.3",
6
7      // Dependencias de librerías
8      libraryDependencies += Seq(
9        // Cats Effect para manejo de efectos
10       "org.typelevel" %% "cats-effect" % "3.5.4",
11
12       // Dependencia de ScalaCheck para testing PBT
13       "org.typelevel" %% "scalacheck" % "1.18.0" % Test,
14
15       // Apache Spark (ej. para ingeniería de datos)
16       "org.apache.spark" %% "spark-sql" % "3.5.1"
17     )
18  )
```

B. Declaración de Módulos (Multi-Project)

Para la modularidad, sbt utiliza `lazy val` para definir cada submódulo, especificando el código fuente y las dependencias inter-módulo.

```
1  // Definición del Proyecto Raíz (que agrega los submódulos)
2  lazy val root = project
3    .in(file("."))
4    .aggregate(core, service, cli) // Agrega todos los módulos
```

```

5
6 // Definición del Módulo 'core'
7 lazy val core = project
8   .in(file("core"))
9   .settings(name := "core-module")
10
11 // Definición del Módulo 'service'
12 lazy val service = project
13   .in(file("service"))
14   // Dependencia: Service depende de Core
15   .dependsOn(core)

```

3. Ejemplos de Código: Desacoplamiento con Traits

El patrón idiomático de Scala para el desacoplamiento fuerte se logra separando la definición de la API (un `trait` abstracto) de su implementación. Si el `Módulo B` necesita usar solo un método de `Módulo A`, definimos una interfaz abstracta en `Módulo B` y usamos la inyección de dependencias (`given` en Scala 3) para enlazar la implementación en tiempo de composición o inicio.

Escenario Realista: Desacoplar el módulo `Reportes` del módulo `BaseDeDatos` para que los cambios en la implementación de `BaseDeDatos` (ej., cambiar de JDBC a Cassandra) no recompilen `Reportes`.

1. Módulo Core/Interfaces (Dependencia Mínima)

En el `core`, definimos solo lo que la lógica de negocio necesita (Ley de Deméter).

```

1 // Definido en el módulo 'Reportes'
2 // Solo declara la función esencial que Reportes necesita del servicio
  'Usuario'.
3 // (Simulamos un trait llamado "Gate" o "Boundary" para el
  desacoplamiento)
4 trait ServicioUsuario {
5   // Solo necesitamos saber si un usuario existe, no todos sus detalles
6   def existe(id: String): Boolean
7 }
8
9 // Clase de negocio que necesita la dependencia inyectada.
10 // En Scala 3, usamos 'using' para inyectar la implementación de
  ServicioUsuario.
11 class GeneradorReporte(using usuarioService: ServicioUsuario) {
12   def generar(userId: String): String =

```

```

13     if usuarioService.existe(userId) then
14         s"Reporte generado para ID: $userId"
15     else
16         s"Error: Usuario $userId no encontrado"
17 }

```

2. Módulo de Implementación (Provee la Lógica)

En el módulo `BaseDeDatos`, se define y proporciona la implementación concreta utilizando un `given` o una `case class` que implementa el *trait*.

```

1  // Definido en el módulo 'BaseDeDatos'
2  import scala.collection.concurrent.TrieMap
3
4  // Implementación real del ServicioUsuario
5  case class ServicioUsuarioImpl() extends ServicioUsuario:
6      // Simulación de una DB en memoria
7      private val usuarios = TrieMap("A42" -> true, "B99" -> false)
8
9      override def existe(id: String): Boolean =
10         usuarios.contains(id)
11
12  // Proveer la instancia canónica de la interfaz (dado/given)
13  // Esto es lo que el compilador inyectará donde se pida 'using
14  // ServicioUsuario'.
15  given servicioUsuario: ServicioUsuario = ServicioUsuarioImpl()

```

Si la implementación de `ServicioUsuarioImpl` cambia (ej., cambia de `TrieMap` a `Doobie` o `Pekko Persistence`), el módulo `Reportes` no requiere cambios en su definición, lo que **minimiza la compilación incremental**.

4. Comparativa con Java o Python

La arquitectura de *tooling* y modularidad de Scala, especialmente en combinación con el tipado estático y sbt, ofrece ventajas específicas en proyectos grandes.

Característica	Java (Maven/Gradle)	Python (Scripts/Monolitos)	Scala (sbt/Scala CLI)
Definición de Dependencias	Archivos externos (XML o Groovy), a	<code>pip</code> (gestión de paquetes), acoplamiento en <i>runtime</i> .	sbt DSL conciso. Scala CLI Directivas para scripts (<code>//></code> <code>using dependency</code>).

Característica	Java (Maven/Gradle)	Python (Scripts/Monolitos)	Scala (sbt/Scala CLI)
	menudo verbosos.		
Compilación Modular	Maven/Gradle soporta módulos, pero la velocidad depende de la configuración.	N/A (Interpretado/Ejecución).	Compilación en Paralelo nativa a través de sbt.
Desacoplamiento	Inyección de dependencias en <i>runtime</i> (Spring, CDI) o uso manual de <code>interface</code> .	Tipado dinámico (seguridad baja).	Desacoplamiento por Tipo (<code>trait</code> y <code>given/using</code>). La inyección se resuelve en tiempo de compilación , reduciendo errores.
Rendimiento de Lógica	Lógica compilada a <i>bytecode</i> JVM.	Penalización por la comunicación entre el intérprete Python y la JVM (ej. PySpark).	Scala UDFs/Lógica Pura se ejecuta como <i>bytecode</i> JVM nativo, optimizado para el alto rendimiento.

5. Best Practices (*The Scala Way*)

- 1. Modularidad Primero con sbt:** Utiliza sbt para dividir grandes aplicaciones en **múltiples submódulos lógicos**. Organiza el código en módulos (`core` , `api` , `persistence`) y utiliza `dependsOn` para definir el grafo de dependencias. Esto maximiza la **compilación incremental** y el paralelismo.
- 2. Abstracción de Dependencias (Law of Demeter):** Implementa el desacoplamiento fuerte asegurándote de que los módulos dependan solo de las **interfaces abstractas (traits)** necesarias, no de las clases de implementación concretas. La inyección se debe gestionar mediante **Abstracciones Contextuales** (`given/using`).
- 3. Tooling Específico:** Usa **sbt** para la gestión de proyectos complejos y **Scala CLI** para *scripts* y tareas rápidas. Para el desarrollo, **Metals** es el IDE recomendado para el entorno moderno de Scala 3.
- 4. Estándares Automatizados:** Incorpora **Scalafix** en el proceso de desarrollo para **forzar estándares de código y evitar la deuda técnica**. Las herramientas de la

metaprogramación (como Scalafix) son fundamentales en la arquitectura moderna de Scala.

5. **Reutilización del `case class`** : La inmutabilidad de los modelos (`case class`) permite que sean compartidos de forma segura a través de los límites de los módulos sin riesgo de concurrencia.

En la arquitectura de Scala, la herramienta de construcción (sbt) no es solo un compilador; es el **arquitecto principal** que define la estructura y el paralelismo del sistema. Al dividir el proyecto en módulos desacoplados por interfaces abstractas, se asegura que los cambios en una sola unidad no derriben ni ralenticen toda la fábrica, permitiendo que la compilación y la evolución sean rápidas y seguras.